

What Should I Cook Now? — Project Documentation

A single self-contained HTML app that turns whatever's in your kitchen into three (or more) real recipe suggestions — generated live by Claude, walked through a short step-by-step order form, and presented as an interactive recipe rail.

1. Overview

The app asks four quick questions (ingredients, time, servings, dietary needs), sends them to Claude with a strict set of rules, and returns three recipes that use *only* what you actually have on hand. From there you can view full recipes in a modal, ask for more suggestions without repeating dishes, and enjoy a warm, food-themed interface with a few playful interactive touches.

Everything — markup, styles, and logic — lives in one `.html` file. No build step, no dependencies beyond two Google Fonts and a live call to Claude.

2. Step-by-Step Order Wizard

Instead of one long form, the order is broken into five short steps:

1. **Ingredients** — free-text box, plus tappable ingredient suggestion chips
2. **Time available** — 15 / 30 / 45 / 60+ minutes
3. **Servings** — 1 / 2 / 4 / 6, or **Custom** (reveals a manual number input for any exact serving count)
4. **Dietary notes** — tappable chips: vegetarian, vegan, gluten-free, dairy-free, low-carb, nut-free
5. **Review** — a summary of everything entered, before firing the order

Navigation & feedback:

- A "leaf-dot" progress indicator shows which step you're on (dots fill in as you go, current step highlighted in the accent color)
 - Back/Next buttons move between steps; Next is disabled/blocked with an inline message if you try to skip ingredients without entering any
 - Moving between steps triggers a **3D flip transition** — the whole order panel rotates edge-on and swaps content mid-flip, rather than a plain fade
-

3. Ingredient Suggestions

Below the order panel sits a separate suggestion panel with tappable ingredient chips (rice, chicken, garlic, tomato, etc.):

- Starts with 16 common ingredients shown

- A "+ **More ingredients**" button reveals 12 more each click, up to a master list of 50 — the button disables itself and reads "No more suggestions" once exhausted
 - Tapping a chip adds it to the ingredients text box (comma-separated); tapping again removes it
 - Manually typing in the ingredients box keeps the chips in sync — if you type "rice" by hand, the Rice chip highlights automatically
 - This panel is intentionally kept **separate** from the flipping order panel — it never flips, so your suggestions stay put and readable while you move through the steps
-

4. Recipe Generation Rules (the AI logic)

Every request to Claude carries a strict prompt contract:

- **Pantry-only constraint:** recipes may only use ingredients the person actually listed, plus four fixed staples (salt, black pepper, cooking oil, water). No invented ingredients, no "if you have X" substitutions — Claude is instructed to double-check its own output against this rule before answering.
 - **Honest "not enough" handling:** if the listed ingredients genuinely can't make a real dish (e.g. just "water"), Claude returns a short, honest explanation instead of fabricating a recipe — the app detects this response shape and shows the message directly rather than an empty or broken card.
 - **Token-budget discipline:** recipes are capped at 8 ingredients and 6 steps (each under 18 words) so responses fit comfortably within the token limit and rarely get cut off.
 - **Resilient parsing:** if a response *does* get cut off mid-JSON (rare, but possible), the app doesn't just error out — it walks the raw text and recovers every complete recipe object it can find, so you might get 2 good recipes instead of a hard failure.
-

5. Recipe Cards & Modal

Each suggested recipe renders as a card showing the title, a time tag, a difficulty tag, and a one-line "why this fits" note.

- **Click any card** to open a modal with the full recipe: ingredients (leaf-bullet list) and numbered steps
- Modal closes via the × button, clicking the backdrop, or pressing **Esc**
- Fully keyboard-accessible: cards are focusable and open the modal on Enter/Space

6. "Want More Recipes?"

A button below the recipe cards lets you keep generating:

- Each click asks Claude for 3 *new* recipes, explicitly instructed not to repeat (or lightly reword) any dish already shown — the growing list of previous titles is fed back into the prompt each time

- New recipes are appended below the existing cards, and the button stays available for another round
- If Claude genuinely runs out of distinct options for the given ingredients, the button reports that instead of quietly repeating itself

7. Visual Design

Color palette (food-app inspired, fully custom to this project):

Role	Color	Hex
Buttons / icons	Deep forest green	 #3A5A40
Panels / banners	Soft sage	 #E9EDC9
App background	Warm cream	 #FEFAE0
Tags / highlights	Terracotta	 #E07A5F
Headings	Dark olive	 #283618
Body / muted text	Muted green-gray	 #588157

Typography: DynaPuff (a rounded, playful display font) for the main title and recipe/modal headings; Work Sans for body text, labels, and lists — chosen because DynaPuff's chunky bubble letterforms work well as short display text but aren't meant for long-form reading.

Signature details:

- A leaf emoji (🍃) marks every ingredient list item
- Numbered steps use small filled circular badges instead of plain numerals
- The header carries a pill-shaped "Order Up" badge with a custom SVG fork/knife icon
- The "Fire the order" button has a slow, breathing glow animation

8. Floating Food Background

A decorative, non-interactive layer of food/vegetable emoji (🍅🥕🍷🍋🍓🍌🍍🍎🍇🍓🍌🍍🍎🍇) animates behind all content, with three distinct behavior modes:

- **Idle** — each icon independently wanders to a new random position on its own long timer (roughly every 9–18 seconds), so the background feels alive without being busy
- **Cursor movement** — the icons gather into a loose ring around the pointer, each holding its own fixed angle and distance so they form a cluster rather than piling up
- **Click** — every icon bursts outward from the click point along its existing angle (just much farther), like a small explosion, then settles back into idle wandering after a few

seconds

All movement is intentionally slow and dreamy rather than snappy. The whole layer sits behind the interface (`pointer-events: none`) so it never interferes with clicks, and it's fully disabled for anyone with `prefers-reduced-motion` set at the OS level.

9. Accessibility & Robustness Notes

- Respects `prefers-reduced-motion`: all CSS animations/transitions are disabled, and the floating background's JS-driven movement is skipped entirely
 - Keyboard support throughout: wizard buttons, ingredient/diet chips, recipe cards, and the modal all work without a mouse
 - Errors are shown in plain language in the results area rather than failing silently (network issues, malformed AI responses, insufficient ingredients, etc.)
-

10. Known Limitation

The recipe-generation call goes directly to Claude's API from the browser. This only works while the file is running inside Claude's own chat preview, which quietly proxies that request. If the file is downloaded and opened directly (or hosted elsewhere) outside that environment, this call will fail with a CORS error, since Claude's public API doesn't accept unauthenticated direct-from-browser requests. Running it standalone would require either a small backend proxy holding a real API key, or wiring in a different provider (e.g. Gemini) that does support direct browser calls with a client-side key.